

# TVHC - Developer Guide

---

Technical handover documentation for AEM Edge  
Delivery Services

**Date:** June 3, 2026

# Table of Contents

- TVHC - Developer Guide ..... 3
  - Quick Reference ..... 3
  - Architecture Overview ..... 3
  - Local Development Setup ..... 5
  - Design System ..... 6
  - Core Script Customizations ..... 7
  - Blocks Reference ..... 8
  - Universal Editor Models ..... 10
  - Common Development Tasks ..... 10
  - Environments ..... 11
  - Git Workflow ..... 11
  - Troubleshooting ..... 11
  - Resources ..... 12
  - Support Contacts ..... 12

# TVHC - Developer Guide

## Quick Reference

| Resource            | URL                                                                                         |
|---------------------|---------------------------------------------------------------------------------------------|
| Code Repository     | <a href="https://dprevelige.tvhc">dprevelige/tvhc</a>                                       |
| Preview             | <a href="https://main-tvhc-dprevelige.aem.page/">https://main-tvhc-dprevelige.aem.page/</a> |
| Live                | <a href="https://main-tvhc-dprevelige.aem.live/">https://main-tvhc-dprevelige.aem.live/</a> |
| Local Dev           | <a href="http://localhost:3000">http://localhost:3000</a>                                   |
| Config Service Org  | dprevelige                                                                                  |
| Reference Demo Docs | <a href="https://adobe.com/go/refdemo">https://adobe.com/go/refdemo</a>                     |

This project extends the **Reference Demo Framework** for AEM Edge Delivery Services — a pharma/healthcare-oriented demo built on the standard boilerplate with Dynamic Media, Content Fragments, adaptive forms, and multi-language support.

## Architecture Overview

### Tech Stack

- Vanilla JavaScript (ES6+ modules)
- CSS3 with custom properties (design tokens)
- No build step for runtime — files served directly from GitHub
- Content from Document Authoring (DA) and AEM Content Fragments
- Universal Editor component models (JSON merged at commit via Husky)
- External integrations: Adobe Launch, Adobe Target, Workfront Fusion webhooks, App Builder API gateway

### Project Structure

```
├── blocks/                # 33 block implementations
├── scripts/
│   ├── aem.js            # Core library (DO NOT MODIFY)
│   ├── scripts.js        # Page decoration, DM image/video logic, E-L-D orchestration
│   ├── delayed.js        # Adobe Target, custom JS, tweetable links
│   ├── utils.js          # Language, path mapping, placeholders, site helpers
│   ├── dom-helpers.js    # DOM element factory helpers
│   ├── slider.js         # Carousel/slider utility
│   └── editor-support-rte.js
├── styles/
│   ├── styles.css        # Critical styles + design tokens
│   ├── fonts.css         # Font Awesome + Adobe Clean fonts
│   ├── lazy-styles.css   # Non-critical global styles
│   └── slider.css
├── models/               # Universal Editor model source files (merged to root JSON)
└── icons/                # SVG icons
```

```
└─ fonts/           # Font assets (Adobe Clean, Font Awesome)
└─ component-*.json # Generated UE config (do not edit directly)
```

There is no `templates/` folder — page structure is driven by metadata and section styling only.

### Three-Phase Loading (E-L-D)

Edge Delivery Services uses a strict Eager-Lazy-Delayed loading strategy to achieve optimal Lighthouse scores. Every file and script must be placed in the correct phase.

| Phase          | Purpose                                                 | What Loads                                                                                     | Performance Impact         |
|----------------|---------------------------------------------------------|------------------------------------------------------------------------------------------------|----------------------------|
| <b>Eager</b>   | Render above-the-fold content as fast as possible (LCP) | <code>styles/styles.css</code> ,<br>first section's blocks,<br><code>scripts/scripts.js</code> | Blocks LCP — keep minimal  |
| <b>Lazy</b>    | Load remaining page content after first paint           | Remaining blocks, header,<br>footer, <code>fonts.css</code> , <code>lazy-styles.css</code>     | Runs after Eager completes |
| <b>Delayed</b> | Load non-essential third-party scripts                  | <code>scripts/delayed.js</code><br>(Adobe Target, custom JS, tweetables)                       | Runs ~3s after page load   |

#### Rules for developers:

- **Never add third-party scripts to `scripts.js`** unless absolutely required — they block LCP. Prefer `delayed.js`.
- **Known exception in this project:** Adobe Launch is currently loaded in `loadEager()`. Consider moving it to `delayed.js` if performance tuning is needed.
- **Never load fonts eagerly** on first visit — `fonts.css` loads lazily (with a desktop/sessionStorage shortcut).
- Blocks in the first section load eagerly; all others load lazily based on DOM position.
- Header and footer load in the lazy phase via `loadHeader()` / `loadFooter()` from `aem.js`.

### Key Files and How They Connect

| File                            | Role                                                                                                                                                                                                                                       | Connects To                                                                            |
|---------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|
| <code>scripts/aem.js</code>     | Core library — <code>loadBlock</code> , <code>loadCSS</code> , <code>decorateBlock</code> , <code>sampleRUM</code> , <code>loadHeader</code> /<br><code>loadFooter</code> , <code>createOptimizedPicture</code> .<br><b>DO NOT MODIFY.</b> | Imported by <code>scripts.js</code> and blocks                                         |
| <code>scripts/scripts.js</code> | Entry point — orchestrates E-L-D phases. Contains Dynamic Media decoration, section backgrounds, data layer, author detection.                                                                                                             | Imports from <code>aem.js</code> , <code>utils.js</code> , <code>dom-helpers.js</code> |
| <code>scripts/delayed.js</code> | Adobe Target, metadata-driven custom JS, tweetable link decoration                                                                                                                                                                         | Imports from <code>aem.js</code> , <code>utils.js</code> , <code>dom-helpers.js</code> |

| File                    | Role                                                                                           | Connects To                                 |
|-------------------------|------------------------------------------------------------------------------------------------|---------------------------------------------|
| scripts/utils.js        | Multi-language helpers, path mapping (paths.json), site name/hostname/DM URL from placeholders | Imported by header, blocks, and scripts     |
| styles/styles.css       | Critical CSS + brand design tokens                                                             | Referenced by all blocks                    |
| blocks/{name}/{name}.js | Block logic — exports default function decorate(block)                                         | Auto-loaded when block class appears in DOM |

**Execution flow:** page load → scripts.js → loadEager() → loadLazy() → loadDelayed() (3000ms timeout, dynamic import of delayed.js)

---

## Local Development Setup

### Prerequisites

- Node.js (no .nvmrc pinned — use current LTS)
- AEM CLI: `npm install -g @adobe/aem-cli`

### Setup Steps

```
git clone https://github.com/dprevelige/tvhc.git
cd tvhc
npm install
aem up
```

### Local Server

- URL: `http://localhost:3000`
- Auto-reload on file changes
- Uses preview content from the configured DA mount by default

This is a **standard single-site setup** (site name: tvhc, org: dprevelige). No `--pages-url` flag is required unless additional sites are added to the same repository via Config Service.

### Linting and JSON Build

```
npm run lint          # ESLint + Stylelint
npm run build:json    # Merge models/*.json into component-*.json
```

The Husky pre-commit hook runs linting and JSON merge automatically.

## Design System

### CSS Custom Properties

Brand tokens are defined at `:root` in `styles/styles.css` and can be overridden per-site via placeholders (e.g., `--brand-theme-color`).

### Typography

```
--body-font-family: 'Adobe Clean', roboto, roboto-fallback, sans-serif;
--heading-font-family: 'Adobe Clean Bold', roboto-condensed, roboto-condensed-fallback,
sans-serif;
--body-font-size-m: 22px; /* scales down at tablet breakpoint */
--heading-font-size-xxl: 55px;
```

### Colors

```
--brand-theme-color: #ff0000;
--brand-dark-color: #131313;
--brand-light-color: #dcdcdc;
--background-color: var(--brand-background-color, #ffffff);
--text-color: var(--brand-text-color, #131313);
--link-hover-color: var(--brand-link-hover-color, #ff0000);
--main-accent-color: var(--brand-theme-color, #FB923C);
--nav-background-color: var(--brand-nav-background-color, #0000008c);
--footer-background-color: var(--brand-footer-background-color, #000);
```

### Spacing and Layout

```
--nav-height: 64px;
--global-gutter: 1.5rem;
--global-section-padding: 5rem;
--spacing-small: 1rem;
--spacing-medium: 2rem;
--spacing-xxlarge: 4rem;
--border-radius-medium: 0.5rem;
```

### Breakpoints

| Name    | Width   | Usage                                            |
|---------|---------|--------------------------------------------------|
| Mobile  | default | Base styles                                      |
| Tablet  | 768px   | Section background images, mobile nav            |
| Desktop | 900px   | Header desktop nav, font eager-load shortcut     |
| Large   | 1024px  | Typography scale-down in <code>styles.css</code> |

Additional project-specific breakpoints appear in section background logic (992px, 768px, 320px).

### Section Styles

Available section modifier classes in `styles/styles.css`:

| Class              | Effect                                               |
|--------------------|------------------------------------------------------|
| .light/ .highlight | Light background treatment                           |
| .bg-theme          | Theme color background                               |
| .bg-dark           | Dark background                                      |
| .bg-light          | Light background                                     |
| .section-has-bg    | Applied when data-image section attribute is present |

Sections can also use data-image, data-tab-image, and data-mob-image attributes for responsive background images (processed in `decorateSectionImages()`).

## Fonts

| Family                                                | Source                       | Usage                      |
|-------------------------------------------------------|------------------------------|----------------------------|
| Adobe Clean (Regular, Bold, Light, Extra Bold, Black) | fonts/ directory             | Body and headings          |
| Font Awesome 5.15.4                                   | styles/fonts.css + fonts/fa/ | Icons in navigation and UI |
| Roboto fallbacks                                      | System/web fallback          | Fallback stack             |

## Core Script Customizations

### scripts/scripts.js

Key exports and functions beyond boilerplate:

| Function                                                           | Purpose                                                                                                                                                                                      |
|--------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>decorateMain()</code>                                        | Calls <code>decorateButtons</code> , <code>decorateIcons</code> , <code>buildAutoBlocks</code> , <code>decorateSections</code> , <code>decorateBlocks</code> , <code>decorateDMImages</code> |
| <code>decorateDMImages()</code>                                    | Transforms Dynamic Media OpenAPI asset links into <code>&lt;picture&gt;</code> elements with smart crop, presets, rotate/flip/crop modifiers                                                 |
| <code>decorateDMVideos()</code>                                    | Renders DM player iframes or HTML5 video for <code>.dynamic-media-video</code> blocks                                                                                                        |
| <code>decorateSectionImages()</code>                               | Builds responsive <code>&lt;picture class="section-bg"&gt;</code> from section data-image attributes                                                                                         |
| <code>renderWBDataLayer()</code>                                   | Populates <code>window.wbgData.page.pageInfo</code> from page metadata                                                                                                                       |
| <code>fetchLanguagePlaceholders()</code>                           | Loads placeholders from <code>/language-masters/{lang}/</code> with English fallback                                                                                                         |
| <code>isAuthorEnvironment()</code>                                 | Detects AEM author/preview context for dual-mode block behavior                                                                                                                              |
| <code>moveAttributes()</code> / <code>moveInstrumentation()</code> | UE instrumentation attribute helpers                                                                                                                                                         |

**Dynamic Media URL pattern:** Links matching `/adobe/assets/urn:aaid:aem:{uuid}` are detected and transformed. Advanced modifiers (smart crop, preset, rotate, flip, crop, extend, background-color) are read from sibling divs in authored content.

#### **scripts/delayed.js**

| Integration     | Details                                                                                                       |
|-----------------|---------------------------------------------------------------------------------------------------------------|
| Adobe Target    | Loads <code>scripts/at-lsig.js</code> with property <code>549d426b-0bcc-be60-ce27-b9923bfcad4f</code>         |
| Custom JS       | Reads <code>js-files</code> page metadata (comma-separated URLs) and loads each via <code>loadScript()</code> |
| Tweetable links | Parses <code>&lt;tweetable&gt;</code> tags in paragraphs and opens Twitter intent popup                       |

All delayed integrations are skipped on `localhost`.

#### **scripts/utils.js**

| Export                                                                  | Purpose                                                                          |
|-------------------------------------------------------------------------|----------------------------------------------------------------------------------|
| <code>PATH_PREFIX</code>                                                | <code>/language-masters</code> — base path for localized content                 |
| <code>SUPPORTED_LANGUAGES</code>                                        | 20 languages (en, fr, de, es, it, pt, nl, sv, da, ru, ja, zh, zh_TW, ko, ar, he) |
| <code>getLanguage() / setPageLanguage()</code>                          | Detect language from URL path segment                                            |
| <code>computeLocalizedUrl()</code>                                      | Build localized URLs for EDS and AEM author paths                                |
| <code>getSiteName() / getHostname() / getDynamicMediaServerURL()</code> | Read from <code>placeholders.json</code>                                         |
| <code>getPathMappings() / mapAemPathToSitePath()</code>                 | Map AEM content paths to site paths via <code>/paths.json</code>                 |
| <code>discoverLanguagesFromPlaceholders()</code>                        | Read available languages from placeholders                                       |
| <code>fetchLanguageNavigation()</code>                                  | Cache navigation JSON per language                                               |

## Blocks Reference

The project contains 33 blocks. The table below covers **project-customized blocks** (multiple commits beyond initial boilerplate setup). Remaining blocks are standard Reference Demo / boilerplate implementations.

| Block          | Purpose                                  | Variants / Config  | Key Features                                                                                                     |
|----------------|------------------------------------------|--------------------|------------------------------------------------------------------------------------------------------------------|
| <b>header</b>  | Site navigation, language switcher, logo | Desktop/mobile nav | Loads nav fragments, 20-language support, tag root <code>wknd-universal</code> , uses <code>navigation.js</code> |
| <b>regform</b> | Event registration form                  | —                  | US state dropdown, field validation, posts to backend, redirects to confirmation page on                         |



| Block                   | Purpose                              | Variants / Config                                                                        | Key Features                                                                                    |
|-------------------------|--------------------------------------|------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
|                         |                                      |                                                                                          | <code>main--tvhc--dprevelige.aem.page</code>                                                    |
| <b>hero</b>             | Hero banner with CTA                 | over lay layout, CTA styles, underline toggle, background style                          | Reads config from block rows; applies <code>cta-{style}</code> classes to button container      |
| <b>cfteaser</b>         | Content Fragment teaser via web-hook | Template name (default: "Top Image CTA")                                                 | Fetches rendered HTML from Workfront Fusion webhook; supports author/publish modes              |
| <b>press-release</b>    | Press release from Content Fragment  | Alignment, variation name                                                                | GraphQL via API gateway wrapper (publish) or direct AEM GraphQL (author)                        |
| <b>textblock</b>        | Rich text with layout control        | <code>textblock-align-{left center right}</code> , <code>textblock-width-{50-100}</code> | Configurable alignment and width percentage                                                     |
| <b>form</b>             | Adaptive Form block                  | Full form component library                                                              | Rule engine, reCAPTCHA, wizard, repeat panels — large subsystem under <code>blocks/form/</code> |
| <b>safetyindication</b> | Pharma safety information            | Expand/collapse                                                                          | Accessible toggle ( <code>aria-expanded</code> ), collapsed by default                          |
| <b>defaultoffer</b>     | Promotional offer card               | Hidden wrapper ( <code>display:none</code> )                                             | Static offer content for NeuroPax demo — image, title, CTA                                      |
| <b>footer</b>           | Site footer                          | —                                                                                        | Customized from boilerplate                                                                     |
| <b>standard</b>         | Standard content block               | —                                                                                        | Minor customization                                                                             |
| <b>teaser</b>           | Teaser block                         | —                                                                                        | Minor customization                                                                             |

## Notable Boilerplate / Framework Blocks

These ship with the Reference Demo Framework and are available but not heavily customized in git history:

| Block                                               | Purpose                                                                                   |
|-----------------------------------------------------|-------------------------------------------------------------------------------------------|
| <code>dynamic-media-image/dynamicmedia-image</code> | Dynamic Media OpenAPI image rendering (also handled globally in <code>scripts.js</code> ) |
| <code>dynamic-media-video</code>                    | DM video player / HTML5 video                                                             |
| <code>dynamicmedia-template</code>                  | DM template rendering                                                                     |
| <code>content-fragment</code>                       | Generic CF display                                                                        |
| <code>find-a-doctor</code>                          | Doctor search with GraphQL, filters, appointment modal                                    |
| <code>cards, carousel, tabs, accordion</code>       | Standard layout blocks                                                                    |
| <code>fragment</code>                               | Loads content fragments from <code>/fragments/</code> paths                               |
| <code>embed-adaptive-form</code>                    | Embeds AEM adaptive forms                                                                 |
| <code>forex</code>                                  | Currency exchange widget                                                                  |

| Block                                                           | Purpose                 |
|-----------------------------------------------------------------|-------------------------|
| video, iframe, quote, columns, search, separator, action-button | Standard utility blocks |

---

## Universal Editor Models

Component models live in `models/` and are merged into root-level JSON on commit:

```
npm run build:json
```

| Source File                                    | Output                                 |
|------------------------------------------------|----------------------------------------|
| <code>models/_component-models.json</code>     | <code>component-models.json</code>     |
| <code>models/_component-definition.json</code> | <code>component-definition.json</code> |
| <code>models/_component-filters.json</code>    | <code>component-filters.json</code>    |

Edit the `models/_*.json` files, not the generated root files.

---

## Common Development Tasks

### Add a New Block

1. Create `blocks/{name}/{name}.js` with `export default function decorate(block)`
2. Create `blocks/{name}/{name}.css`
3. Optionally add `blocks/{name}/_ {name}.json` for Universal Editor
4. Test locally at `http://localhost:3000`
5. Push to a feature branch and verify preview URL

### Modify Global Styles

1. Edit design tokens in `styles/styles.css :root`
2. Test across mobile, tablet, and desktop breakpoints
3. Watch for CLS impact when changing spacing or font sizes

### Add Analytics or Marketing Tool

1. Add to `scripts/delayed.js`
2. Never add to `loadEager()` unless render-critical
3. Verify delayed loading in Network tab (should appear ~3s after load)

### Work with Content Fragments

Blocks like `cfteaser` and `press-release` fetch CF data via:

- **Author:** Direct AEM GraphQL against `authorurl` metadata
- **Publish:** App Builder API gateway at `3635370-refdemoapigateway-stage.adobeioruntime.net`

Ensure page metadata includes `hostname`, `authorurl`, and correct GraphQL endpoint paths.

## Debug a Block

1. Check browser console for fetch/GraphQL errors
2. Inspect DOM: authored table structure vs. decorated output
3. Verify variant/config classes are applied
4. For author-only features, confirm `isAuthorEnvironment()` returns expected value

---

## Environments

| Environment    | URL Pattern                                            | Purpose     |
|----------------|--------------------------------------------------------|-------------|
| Local          | <code>http://localhost:3000</code>                     | Development |
| Feature Branch | <code>https://{branch}-tvhc-dprevelige.aem.page</code> | PR testing  |
| Preview        | <code>https://main-tvhc-dprevelige.aem.page</code>     | Staging     |
| Live           | <code>https://main-tvhc-dprevelige.aem.live</code>     | Production  |

---

## Git Workflow

### Branch Naming

- Features: `feature/description`
- Fixes: `fix/description`
- Keep branch names short (Helix URL length limits)

### PR Requirements

1. Include preview URL in PR description
2. Pass `npm run lint`
3. Test on feature branch preview
4. Verify no console errors on key pages
5. UE JSON files regenerate via pre-commit hook — commit the merged output

---

## Troubleshooting

### Block not loading?

1. Confirm block folder name matches CSS class name exactly
2. Verify JS exports `default function decorate`
3. Check Network tab for 404 on block JS/CSS

### Dynamic Media images not rendering?

1. Confirm link matches `/adobe/assets/urn:aaid:aem:{uuid}` pattern
2. Block must be inside `.dm-openapi` or `.dynamic-media-image` container
3. Check smart crop metadata fetch if `enableSmartCrop` is authored

### Content Fragment blocks empty?

1. Verify GraphQL endpoint and CF path in authored content

2. Check API gateway response in Network tab (publish)
3. Confirm hostname and authorurl metadata are set on the page

### Styles not applying?

1. Check CSS specificity and block class names
2. Verify CSS file loads in Network tab
3. Run `npm run lint:css` for syntax errors

### Content not updating?

1. Clear browser cache
2. Trigger preview in Document Authoring
3. Allow 1–2 minutes for CDN cache propagation

---

## Resources

| Resource                             | URL                                                                                                                           |
|--------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| Edge Delivery Services Documentation | <a href="https://www.aem.live/docs/">https://www.aem.live/docs/</a>                                                           |
| Developer Tutorial                   | <a href="https://www.aem.live/developer/tutorial">https://www.aem.live/developer/tutorial</a>                                 |
| Block Collection                     | <a href="https://www.aem.live/developer/block-collection">https://www.aem.live/developer/block-collection</a>                 |
| E-L-D Loading                        | <a href="https://www.aem.live/developer/keeping-it-100">https://www.aem.live/developer/keeping-it-100</a>                     |
| Best Practices                       | <a href="https://www.aem.live/docs/dev-collab-and-good-practices">https://www.aem.live/docs/dev-collab-and-good-practices</a> |
| Reference Demo Framework             | <a href="https://adobe.com/go/refdemo">https://adobe.com/go/refdemo</a>                                                       |
| Config Service Admin API             | <a href="https://www.aem.live/docs/admin.html">https://www.aem.live/docs/admin.html</a>                                       |

---

## Support Contacts

Add project-specific contacts for development, content, and operations teams as needed.